

Progetto

Importate su Splunk i dati di esempio "tutorialdata.zip":

- Crea una query Splunk per identificare tutti i tentativi di accesso falliti "Failed password". La query dovrebbe mostrare il timestamp, l'indirizzo IP di origine, il nome utente e il motivo del fallimento.
- Scrivi una query Splunk per trovare tutte le sessioni SSH aperte con successo. La query dovrebbe filtrare per l'utente "djohnson" e mostrare il timestamp e l'ID utente.
- Scrivi una query Splunk per trovare tutti i tentativi di accesso falliti provenienti dall'indirizzo IP "86.212.199.60". La query dovrebbe mostrare il timestamp, il nome utente e il numero di porta.
- Crea una query Splunk per identificare gli indirizzi IP che hanno tentato di accedere ("Failed password") al sistema più di 5 volte. La query dovrebbe mostrare l'indirizzo IP e il numero di tentativi.
- Crea una query Splunk per trovare tutti gli Internal Server Error.

Trarre delle conclusioni sui log analizzati utilizzando AI.

Il progetto presentatoci da Epicode, nonché ultimo, ci richiede di esportare ed analizzare dei dati su "Splunk", dandoci alcuni punti da seguire.

Ma cos'è Splunk ?

Splunk è una piattaforma di analisi e monitoraggio che serve per raccogliere, indicizzare e analizzare grandi volumi di dati generati dai sistemi IT, applicazioni e dispositivi in tempo reale.

In pratica, prende i log e altri dati grezzi, li organizza in modo consultabile, e ti permette di fare ricerche, creare dashboard e generare avvisi.

A cosa serve Splunk ?

- Monitoraggio in tempo reale: controllare lo stato di server, applicazioni e reti.
- Analisi dei log: capire cosa è successo e perché (troubleshooting).
- Sicurezza: rilevare anomalie e potenziali attacchi (SIEM).
- Reportistica: creare dashboard interattive e grafici.
- Compliance: conservare i log per motivi legali o normativi.

Una volta capito a cosa serve e cos'è iniziamo a svolgere i punti richiesti

1. **Crea una query Splunk per identificare tutti i tentativi di accesso falliti "Failed password". La query dovrebbe mostrare il timestamp, l'indirizzo IP di origine, il nome utente e il motivo del fallimento.**

Cos'è una Query ?

Una query è semplicemente una domanda (istruzione) che fai a un sistema per ottenere delle informazioni dai dati che contiene.

In informatica, una query dice

"Cercami queste cose, in questo modo, da questi dati".

Una fatta questa doverosa spiegazione possiamo al punto "uno"

```
index=* "Failed password"
| rex field=_raw "Failed password for (?:(?<invalid>invalid user)\s+)?(?<user>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3}) port (?<dest_port>\d+)"
| eval motivo=if(isnotnull(invalid),"utente inesistente","password errata")
| table _time, src_ip, user, dest_port, motivo, _raw
| sort _time
```

Questa e la Query usata per la ricerca degli accessi falliti

_time	src_ip	user	dest_port	motivo	_raw
2025-08-03 13:06:50	27.1.11.11	info	2030	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[5092]: Failed password for invalid user info from 27.1.11.11 port 2030 ssh2
2025-08-03 13:06:50	27.1.11.11	helpdesk	2052	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[3793]: Failed password for invalid user helpdesk from 27.1.11.11 port 2052 ssh2
2025-08-03 13:06:50	27.1.11.11	system	1104	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[2954]: Failed password for invalid user system from 27.1.11.11 port 1104 ssh2
2025-08-03 13:06:50	27.1.11.11	db4	2030	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[5985]: Failed password for invalid user db4 from 27.1.11.11 port 2030 ssh2
2025-08-03 13:06:50	74.125.19.106	yp	1059	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[1504]: Failed password for invalid user yp from 74.125.19.106 port 1059 ssh2
2025-08-03 13:06:50	74.125.19.106	shoutcast	3088	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[1419]: Failed password for invalid user shoutcast from 74.125.19.106 port 3088 ssh2
2025-08-03 13:06:50	74.125.19.106	oracle	4811	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[3411]: Failed password for invalid user oracle from 74.125.19.106 port 4811 ssh2
2025-08-03 13:06:50	74.125.19.106	postgres	1328	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[1874]: Failed password for invalid user postgres from 74.125.19.106 port 1328 ssh2
2025-08-03 13:06:50	74.125.19.106	madonna	3289	password errata	Thu Aug 03 2025 13:06:50 www1 sshd[4204]: Failed password for madonna from 74.125.19.106 port 3289 ssh2
2025-08-03 13:06:50	74.125.19.106	informix	3213	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[4930]: Failed password for invalid user informix from 74.125.19.106 port 3213 ssh2
2025-08-03 13:06:50	74.125.19.106	nginx	2059	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[2882]: Failed password for invalid user nginx from 74.125.19.106 port 2059 ssh2
2025-08-03 13:06:50	74.125.19.106	postgres	3910	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[2571]: Failed password for invalid user postgres from 74.125.19.106 port 3910 ssh2
2025-08-03 13:06:50	74.125.19.106	jira	1227	password errata	Thu Aug 03 2025 13:06:50 www1 sshd[2058]: Failed password for jira from 74.125.19.106 port 1227 ssh2
2025-08-03 13:06:50	74.125.19.106	zabbix	4690	utente inesistente	Thu Aug 03 2025 13:06:50 www1 sshd[1493]: Failed password for invalid user zabbix from 74.125.19.106 port 4690 ssh2

Come possiamo notare Splunk ci divide automaticamente la data, l'user, la porta e il motivo del fallimento.

2. Scrivi una query Splunk per trovare tutte le sessioni SSH aperte con successo. La query dovrebbe filtrare per l'utente "djohnson" e mostrare il timestamp e l'ID utente.

```
index=* ("Accepted password" OR "Accepted publickey") "for djohnson"
| rex field=_raw "Accepted \S+ for (?<user>djohnson) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3}) port (?<dest_port>\d+)"
| table _time, user, src_ip, dest_port, _raw
| sort _time
```

_time ↕	user ↕ ✓	src_ip ↕ ✓	dest_port ↕ ✓	_raw ↕
2025-08-03 13:06:50	djohnson	10.3.10.46	2925	Thu Aug 03 2025 13:06:50 www1 sshd[47799]: Accepted password for djohnson from 10.3.10.46 port 2925 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	2542	Thu Aug 03 2025 13:06:50 www1 sshd[78112]: Accepted password for djohnson from 10.3.10.46 port 2542 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	2188	Thu Aug 03 2025 13:06:50 www1 sshd[13967]: Accepted password for djohnson from 10.3.10.46 port 2180 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	2662	Thu Aug 03 2025 13:06:50 www1 sshd[93063]: Accepted password for djohnson from 10.3.10.46 port 2662 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	5195	Thu Aug 03 2025 13:06:50 www1 sshd[19864]: Accepted password for djohnson from 10.3.10.46 port 5195 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	6814	Thu Aug 03 2025 13:06:50 www1 sshd[80826]: Accepted password for djohnson from 10.3.10.46 port 6814 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	9811	Thu Aug 03 2025 13:06:50 www1 sshd[40608]: Accepted password for djohnson from 10.3.10.46 port 9811 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	2894	Thu Aug 03 2025 13:06:50 www1 sshd[54378]: Accepted password for djohnson from 10.3.10.46 port 2894 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	9931	Thu Aug 03 2025 13:06:50 www1 sshd[10821]: Accepted password for djohnson from 10.3.10.46 port 9931 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	6927	Thu Aug 03 2025 13:06:50 www1 sshd[70527]: Accepted password for djohnson from 10.3.10.46 port 6927 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	2102	Thu Aug 03 2025 13:06:50 www1 sshd[36970]: Accepted password for djohnson from 10.3.10.46 port 2102 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	5000	Thu Aug 03 2025 13:06:50 www1 sshd[53619]: Accepted password for djohnson from 10.3.10.46 port 5000 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	9171	Thu Aug 03 2025 13:06:50 www1 sshd[33951]: Accepted password for djohnson from 10.3.10.46 port 9171 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	9210	Thu Aug 03 2025 13:06:50 www1 sshd[50044]: Accepted password for djohnson from 10.3.10.46 port 9210 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	3491	Thu Aug 03 2025 13:06:50 www1 sshd[84484]: Accepted password for djohnson from 10.3.10.46 port 3491 ssh2
2025-08-03 13:06:50	djohnson	10.3.10.46	9177	Thu Aug 03 2025 13:06:50 www1 sshd[6069]: Accepted password for djohnson from 10.3.10.46 port 9177 ssh2
2025-08-03 13:06:51	djohnson	10.3.10.46	1991	Thu Aug 03 2025 13:06:51 www3 sshd[57127]: Accepted password for djohnson from 10.3.10.46 port 1991 ssh2
2025-08-03 13:06:51	djohnson	10.3.10.46	5902	Thu Aug 03 2025 13:06:51 www3 sshd[68547]: Accepted password for djohnson from 10.3.10.46 port 5902 ssh2
2025-08-03 13:06:51	djohnson	10.3.10.46	6597	Thu Aug 03 2025 13:06:51 www3 sshd[3902]: Accepted password for djohnson from 10.3.10.46 port 6597 ssh2
2025-08-03 13:06:51	djohnson	10.3.10.46	1921	Thu Aug 03 2025 13:06:51 www3 sshd[81250]: Accepted password for djohnson from 10.3.10.46 port 1921 ssh2

Come possiamo vedere dalle immagini sopra elencate la Query ci serve per identificare il numero di sessioni SSH aperte con successo e l'indirizzo IP dalla quale arrivano le richieste e il numero di tentativi in questo caso dall'utente "djhonson"

3. Scrivi una query Splunk per trovare tutti i tentativi di accesso falliti provenienti dall'indirizzo IP "86.212.199.60". La query dovrebbe mostrare il timestamp, il nome utente e il numero di porta.

```
index=* "Failed password"
| rex field=_raw "Failed password for (?:(?:invalid user)\s+)?(?:<user>\s+) from (?<src_ip>\d{1,3}?(?:\.\d{1,3}){3}) port (?<dest_port>\d+)"
| search src_ip="86.212.199.60"
| table _time, user, dest_port, _raw
| sort _time
```

_time ↕	user ↕ ✓	dest_port ↕ ✓	_raw ↕
2025-08-03 13:06:52	hsqldb	4379	Thu Aug 03 2025 13:06:52 www2 sshd[3218]: Failed password for invalid user hsqldb from 86.212.199.60 port 4379 ssh2
2025-08-03 13:06:52	admin	2831	Thu Aug 03 2025 13:06:52 www2 sshd[3035]: Failed password for invalid user admin from 86.212.199.60 port 2831 ssh2
2025-08-03 13:06:52	system	4294	Thu Aug 03 2025 13:06:52 www2 sshd[1283]: Failed password for invalid user system from 86.212.199.60 port 4294 ssh2
2025-08-03 13:06:52	alex	2121	Thu Aug 03 2025 13:06:52 www2 sshd[2308]: Failed password for invalid user alex from 86.212.199.60 port 2121 ssh2
2025-08-03 13:06:52	mantis	2735	Thu Aug 03 2025 13:06:52 www2 sshd[4525]: Failed password for invalid user mantis from 86.212.199.60 port 2735 ssh2
2025-08-03 13:06:52	jira	1958	Thu Aug 03 2025 13:06:52 www2 sshd[3886]: Failed password for jira from 86.212.199.60 port 1958 ssh2
2025-08-03 13:06:52	vmware	4457	Thu Aug 03 2025 13:06:52 www2 sshd[4205]: Failed password for invalid user vmware from 86.212.199.60 port 4457 ssh2
2025-08-03 13:06:52	nagios	2048	Thu Aug 03 2025 13:06:52 www2 sshd[1937]: Failed password for nagios from 86.212.199.60 port 2048 ssh2
2025-08-03 13:06:52	mail	1775	Thu Aug 03 2025 13:06:52 www2 sshd[1843]: Failed password for mail from 86.212.199.60 port 1775 ssh2
2025-08-03 13:06:53	shoutcast	2227	Thu Aug 03 2025 13:06:53 mailsv1 sshd[3665]: Failed password for invalid user shoutcast from 86.212.199.60 port 2227 ssh2
2025-08-03 13:06:53	nagios	3362	Thu Aug 03 2025 13:06:53 mailsv1 sshd[2324]: Failed password for nagios from 86.212.199.60 port 3362 ssh2
2025-08-03 13:06:53	oracle	2411	Thu Aug 03 2025 13:06:53 mailsv1 sshd[3063]: Failed password for invalid user oracle from 86.212.199.60 port 2411 ssh2
2025-08-03 13:06:53	noone	2381	Thu Aug 03 2025 13:06:53 mailsv1 sshd[5985]: Failed password for invalid user noone from 86.212.199.60 port 2381 ssh2
2025-08-03 13:06:53	email	2213	Thu Aug 03 2025 13:06:53 mailsv1 sshd[1515]: Failed password for invalid user email from 86.212.199.60 port 2213 ssh2
2025-08-03 13:06:53	testuser	1622	Thu Aug 03 2025 13:06:53 mailsv1 sshd[1059]: Failed password for invalid user testuser from 86.212.199.60 port 1622 ssh2
2025-08-03 13:06:53	dean	1136	Thu Aug 03 2025 13:06:53 mailsv1 sshd[1082]: Failed password for invalid user dean from 86.212.199.60 port 1136 ssh2
2025-08-03 13:06:53	desktop	4564	Thu Aug 03 2025 13:06:53 mailsv1 sshd[2554]: Failed password for invalid user desktop from 86.212.199.60 port 4564 ssh2
2025-08-03 13:06:53	mantis	1608	Thu Aug 03 2025 13:06:53 mailsv1 sshd[5140]: Failed password for invalid user mantis from 86.212.199.60 port 1608 ssh2
2025-08-03 13:06:53	root	2753	Thu Aug 03 2025 13:06:53 mailsv1 sshd[2797]: Failed password for root from 86.212.199.60 port 2753 ssh2
2025-08-03 13:06:53	operator	1782	Thu Aug 03 2025 13:06:53 mailsv1 sshd[3011]: Failed password for invalid user operator from 86.212.199.60 port 1782 ssh2

La Query nelle immagini invece ci indicherà tutti i tentativi falliti provenienti dal IP "86.212.199.60".

4. Crea una query Splunk per identificare gli indirizzi IP che hanno tentato di accedere ("Failed password") al sistema più di 5 volte. La query dovrebbe mostrare l'indirizzo IP e il numero di tentativi.

```
index=* "Failed password"
| rex field=_raw "Failed password for (?:(?:invalid user)\s+)?(?<user>\S+) from (?<src_ip>\d{1,3}(\.?\d{1,3}){3})"
| stats count as tentativi, dc(user) as utenti_unici, values(user) as utenti, min(_time) as first, max(_time) as last by src_ip
| where tentativi > 5
| convert ctime(first) ctime(last)
| table src_ip, tentativi, utenti_unici, utenti, first, last
| sort - tentativi
```

src_ip	tentativi	utenti_unici	utenti	first	last
87.194.216.51	948	140	abc	08/03/2025 13:06:50	08/10/2025 13:06:53
211.166.11.101	743	135	adm admin administrator alex	08/03/2025 13:06:50	08/10/2025 13:06:53
128.241.220.82	622	129	adm admin administrator alex amanda	08/03/2025 13:06:50	08/10/2025 13:06:53

La ricerca del quarto punto ci mostrerà gli ip che hanno effettuato tentativi di accesso superiori alle 5 volte, io ne ho elencati solo alcuni, cioè quelli con tentativi di accesso davvero sospetti come si può vedere dal grafico presente in immagine.

5. Crea una query Splunk per trovare tutti gli Internal Server Error.

```
index=* ("Internal Server Error" OR status=500 OR " 500 ")
| rex field=_raw "(?<clientip>\S+) \S+ \S+ \[^\]]+\] \"(?<method>\S+) (?<uri_path>\S+)[^\"]*\" (?<status>\d{3}) (?<bytes>\S+)"
| search status=500
| table _time, clientip, method, uri_path, status, bytes, _raw
| sort _time
```

_time	clientip	method	uri_path	status	bytes	_raw
2025-08-03 18:34:11	125.89.78.6	GET	/cart.do?action=remove&itemId=EST-27&JSESSIONID=S08SL10FF1ADFF5003	500	262	125.89.78.6 - - [03/Aug/2025:18:34:11] "GET /cart.do?action=remove&itemId=EST-27&JSESSIONID=S08SL10FF1ADFF5003 HTTP/1.1" 500 262 "http://www.buttercupgames.com/product.screen?productId=SF-BVS-G01" "Mozilla/5.0 (iPad; CPU OS 5_1_1 like Mac OS X) AppleWebKit/534.46 (KHTML, like Gecko) Version/5.1 Mobile/9B206 Safari/7534.48.3" 560
2025-08-03 18:34:12	125.89.78.6	GET	/cart.do?action=purchase&itemId=EST-17&JSESSIONID=S08SL10FF1ADFF5003	500	2042	125.89.78.6 - - [03/Aug/2025:18:34:12] "GET /cart.do?action=purchase&itemId=EST-17&JSESSIONID=S08SL10FF1ADFF5003 HTTP/1.1" 500 2042 "http://www.buttercupgames.com/product.screen?productId=SF-BVS-G01" "Mozilla/5.0 (iPad; CPU OS 5_1_1 like Mac OS X) AppleWebKit/534.46 (KHTML, like Gecko) Version/5.1 Mobile/9B206 Safari/7534.48.3" 852
2025-08-03 18:34:14	125.89.78.6	GET	/oldlink?itemId=EST-16&JSESSIONID=S08SL10FF1ADFF5003	500	570	125.89.78.6 - - [03/Aug/2025:18:34:14] "GET /oldlink?itemId=EST-16&JSESSIONID=S08SL10FF1ADFF5003 HTTP/1.1" 500 570 "http://www.buttercupgames.com/product.screen?productId=SF-BVS-G01" "Mozilla/5.0 (iPad; CPU OS 5_1_1 like Mac OS X) AppleWebKit/534.46 (KHTML, like Gecko) Version/5.1 Mobile/9B206 Safari/7534.48.3" 265
2025-08-03 18:42:26	195.69.168.22	GET	/oldlink?itemId=EST-7&JSESSIONID=S02SL10FF1ADFF5054	500	2538	195.69.168.22 - - [03/Aug/2025:18:42:26] "GET /oldlink?itemId=EST-7&JSESSIONID=S02SL10FF1ADFF5054 HTTP/1.1" 500 2538 "http://www.buttercupgames.com/product.screen?productId=SF-BVS-G01" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 685
2025-08-03 18:53:54	69.175.97.11	GET	/product.screen?productId=SF-BVS-G01&JSESSIONID=S08SL9FF9ADFF5091	500	837	69.175.97.11 - - [03/Aug/2025:18:53:54] "GET /product.screen?productId=SF-BVS-G01&JSESSIONID=S08SL9FF9ADFF5091 HTTP/1.1" 500 837 "http://www.buttercupgames.com/product.screen?productId=SF-BVS-G01" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_8) AppleWebKit/534.55.3 (KHTML, like Gecko) Version/5.1.5 Safari/534.55.3" 734
2025-08-03 19:07:21	203.92.58.136	GET	/product.screen?productId=SF-BVS-G01&JSESSIONID=S08SL9FF9ADFF5158	500	2257	203.92.58.136 - - [03/Aug/2025:19:07:21] "GET /product.screen?productId=SF-BVS-G01&JSESSIONID=S08SL9FF9ADFF5158 HTTP/1.1" 500 2257 "http://www.buttercupgames.com/cart.do?action=addtocart&itemId=EST-7" "Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.9.2.28) Gecko/20120306 Firefox/3.6.28 C...NET CLR 3.5.30729; .NET4.0C" 948

Con la Query presentata andremo a trovare tutti gli “**internal server error**”

Analisi e Conclusioni sui Log

1. Tentativi di accesso falliti (“Failed password”)

- Volume rilevante: oltre 33.000 eventi di accesso SSH fallito.
- Distribuzione motivi:
 - Prevalenza di *utente inesistente*: indica tentativi di enumerazione account (ricerca di nomi utente validi).
 - Una parte minore di *password errata* su utenti esistenti: indica tentativi di brute-force mirato.
- Origine geografica potenziale: la varietà di IP sorgenti (es. 27.1.11.11, 74.125.19.106, ecc.) indica probabile uso di botnet o sistemi distribuiti.
- Porte di destinazione non standard: oltre alla porta SSH predefinita (22/2222), troviamo tentativi verso porte elevate (2030, 3088, ecc.), segno di scansione per SSH su porte alternative.

Rischio: Alto – attività tipica di attacchi automatizzati di forza bruta e reconnaissance.

2. Attività su utente specifico (djohnson)

- Nessuna anomalia evidente in numero di sessioni riuscite (da verificare con query).
- Se si osservano accessi riusciti da IP non aziendali → potenziale compromissione credenziali.

3. IP sorgente mirato: 86.212.199.60

- Più tentativi di accesso fallito verso diversi account e porte.
- Attività concentrata in determinate finestre temporali → suggerisce scansione manuale o script schedulato.

4. IP con più di 5 tentativi falliti

- Diversi IP superano la soglia, alcuni con centinaia di tentativi → forte indicatore di bruteforce.

- Molti IP appaiono solo per poche ore, segno di rotazione IP (evasion tecnica comune).

5. Internal Server Error (HTTP 500)

- Presenza di HTTP 500 indica possibili malfunzionamenti applicativi o input non validi inviati al server.
- Se correlati a picchi di tentativi SSH → ipotesi di test multi-vettore (attacco web + SSH).

Sintesi valutativa

I log mostrano un pattern coerente con:

1. Reconnaissance: enumerazione utenti e porte.
2. Bruteforce SSH: migliaia di tentativi distribuiti nel tempo e nello spazio IP.
3. Possibili test su applicazioni web: errori 500 che potrebbero derivare da input malevoli o fuzzing.

Raccomandazioni

- Protezione SSH:
 - Abilitare MFA.
 - Limitare login SSH solo a IP trusted (whitelist).
 - Ridurre tentativi consentiti (MaxAuthTries).
 - Implementare Fail2Ban o CrowdSec.
- Protezione Web:
 - Monitorare e correggere cause degli HTTP 500.
 - Applicare WAF per filtrare richieste anomale.
- Monitoraggio & Alerting:
 - Alert in tempo reale per >5 tentativi falliti in 10 min da stesso IP.
 - Alert per picchi di errori HTTP 500.
- Log intelligence:

- Integrare con feed IP malevoli per blocco proattivo.
- Automatizzare report settimanali di trend attacchi.